



?

Too low

Too high

Correct!

GROUP TWO

A Guess Number Game

Expanded Challenge: a command-line number guessing game in Python

[Levels](#) · [Algorithm](#) · [Modules](#) · [game\(\)](#) · [Scoring](#) · [Leaderboard](#) · [Q&A](#)

Contents

1 Project Overview	3
2 Introduction	3
3 Project Progression: Level 1–4	5
4 General Algorithm of the Project	7
5 Python Modules Used	7
6 Functions Created Before game()	9
7 The game() Function	12
8 Detailed Code Walkthrough	15
9 Technical Concepts Used	19
10 Game State and Data Flow	20
11 Single-Player and Multiplayer: One Logic	22
12 Hint System	22
13 Scoring and Rating System	22
14 Leaderboard System	23
15 Level 1–3 Implementation Reference	24
16 Important Questions and Answers	26
17 Conclusion	27

1

Project Overview

Item	Detail
Project title	Number Guessing Game — Expanded Challenge
Project type	Command-Line Interface (CLI) Python application
Group	Group 2
Final version	Contained in the <code>game()</code> function; Level 4 of the project

Project goal. The project is a number guessing game in which a computer generates a secret number and one or more players attempt to guess it. The game progressively introduces programming concepts, and the final implementation brings all of them together into one complete game system.

Concepts introduced along the way:

- random number generation
- functions
- loops
- conditional statements
- input validation
- exception handling
- difficulty levels
- attempt management
- hints
- scoring
- multiplayer turn management
- persistent data storage
- leaderboard management

How to read this document

Sections 1–4 explain the idea and architecture. Sections 5–6 cover the modules and helper functions. Sections 7–8 explain `game()` in detail. Sections 9–14 cover state, scoring, hints and the leaderboard. Section 15 keeps the Level 1–3 code as reference, and Section 16 collects questions and answers.

2

Introduction

A number guessing game is a simple programming problem in which the computer selects a number that the player does not know. The player repeatedly enters guesses, and the program compares each guess with the secret number and gives feedback:

Feedback	Meaning
Too high	The player's guess is greater than the secret number.
Too low	The player's guess is smaller than the secret number.
Correct	The player's guess matches the secret number.

Although the basic idea is simple, the project was designed to move from a one-time guess to a more complete game containing difficulty selection, limited attempts, input validation, hints, scoring, player ratings, multiplayer support, multiple rounds and leaderboard persistence.

The final version uses a **modular design**: different tasks are separated into different functions. The central function is `game()`. It coordinates the entire application and calls the other functions whenever their functionality is needed.

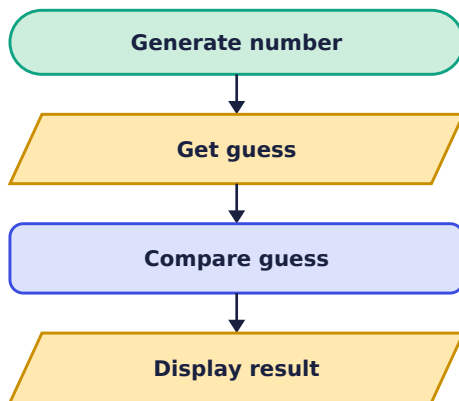
3

Project Progression: Level 1-4

The project was designed in four levels representing the gradual development of the same basic idea.

Level 1 — Basic

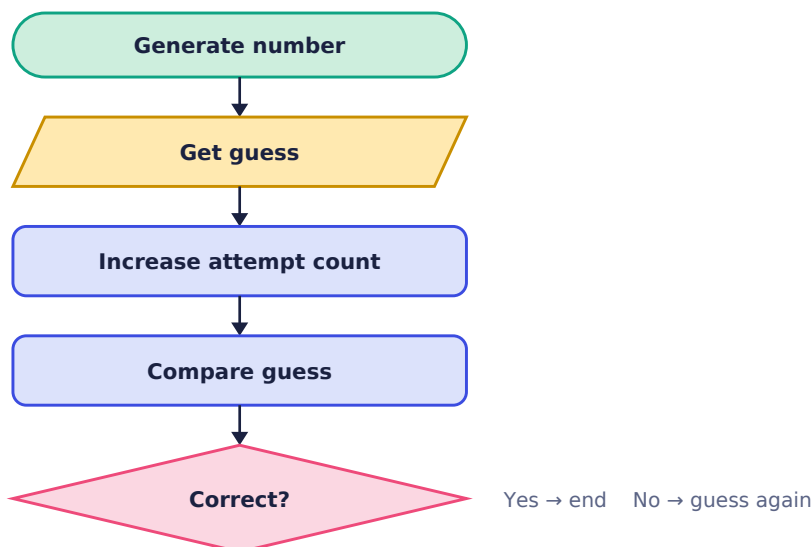
The first level introduces the fundamental concept. The program generates a random number, gets one guess from the player, compares the guess with the secret number and displays the result.



At this stage there is no repeated guessing, scoring, multiplayer system or leaderboard.

Level 2 — Intermediate

Level 2 introduces repetition. Instead of allowing only one guess, the player can continue guessing until the correct number is found. A counter is also introduced to track attempts.



This level introduces `while` loops, counters, repeated input and termination conditions.

Level 3 — Problem Solving

Level 3 makes the game more realistic. The player selects a difficulty, and the program validates input.

Difficulty	Range	Attempts
Easy	1-50	5
Medium	1-100	7
Hard	1-500	8

If the player enters `abc`, the program does not crash. If the player enters `700` while the range is 1-500, the program rejects the guess. Level 3 therefore introduces input validation, `try/except`, difficulty scaling, maximum attempts and game-over conditions.

Level 4 — Expanded Challenge

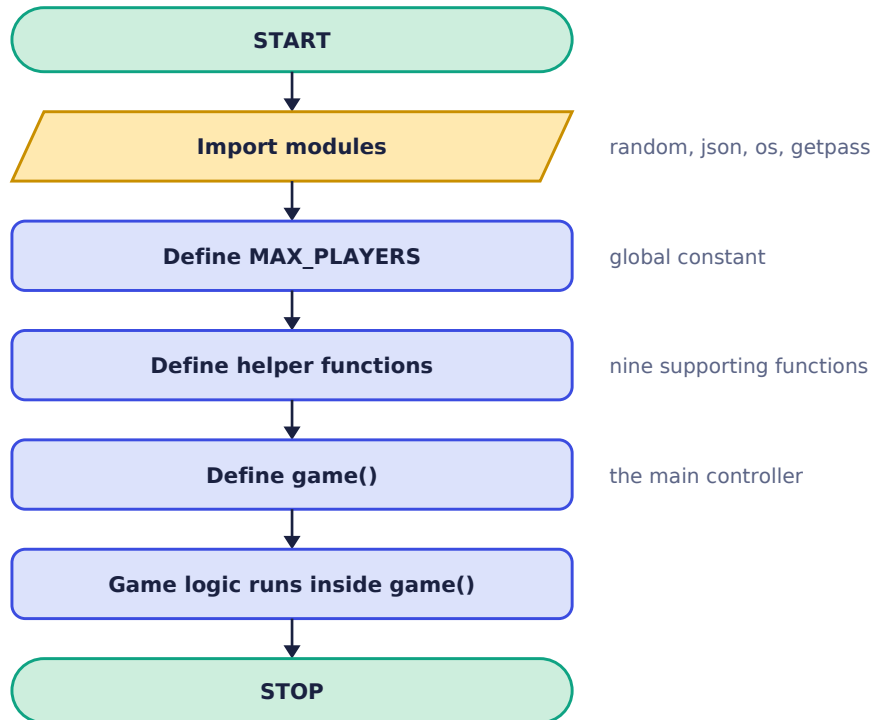
Level 4 combines the previous concepts and adds seven features:

#	Feature	Description
1	Hint system	After every three full rounds of wrong guesses, the player receives a hint.
2	Scoring system	The fewer attempts used, the higher the score.
3	Rating system	The score is converted into a star rating.
4	Multiplayer mode	Multiple players take turns guessing the same secret number.
5	Multiple rounds	Players continue with new rounds without restarting the program.
6	Leaderboard	The program stores each player's best score.
7	Persistent storage	The leaderboard lives in a JSON file and is loaded again when the program runs.

4

General Algorithm of the Project

This is the general algorithm of the entire program, not the detailed algorithm of `game()`. Its purpose is to show the overall architecture of the application.



Helper functions defined before `game()`

<code>generate_secret_number()</code>	<code>compare_guess()</code>	<code>choose_difficulty()</code>
<code>get_valid_guess()</code>	<code>calculate_score()</code>	<code>get_rating()</code>
<code>choose_game_mode()</code>	<code>give_hint()</code>	<code>display_leaderboard()</code>

Important distinction

The functions above `game()` do not independently run the entire game. They are helper functions that perform specific tasks `game()` needs. `game()` is the main controller: it coordinates those functions and determines the order in which the game operates.

Function definitions create reusable instructions; they do not execute those instructions immediately. The program only starts running the game when `game()` is called.

5

Python Modules Used

The program imports four modules:

```
import random
import json
import os
import getpass
```

A **module** is a Python file or library containing code that provides functionality another program can reuse. Instead of implementing everything ourselves, Python lets us import existing

functionality.

5.1 random

The `random` module generates pseudo-random values. The program uses `random.randint(low, high)`, which returns a random integer between `low` and `high`, **including both boundaries**.

```
random.randint(1, 100) # could produce 37, 82, 1 or 100
```

Why we use it: the game needs a secret number the player cannot predict.

5.2 json

JSON stands for **JavaScript Object Notation**, a common text-based format for storing and exchanging structured data. The game uses it to store the leaderboard.

```
{
  "Mathias": 90,
  "John": 75
}
```

A Python dictionary can be converted into JSON and saved to a file. `json.dump()` writes Python data into JSON; `json.load()` reads JSON back into Python.

5.3 os

The `os` module lets Python interact with operating-system features. The program uses it for two purposes:

Purpose	Code	Notes
Check whether a file exists	<code>os.path.exists(leaderboard_file)</code>	Returns True or False.
Clear the terminal	<code>os.system("cls")</code> or <code>os.system("clear")</code>	<code>cls</code> is generally for Windows; <code>clear</code> is for Linux and macOS.

5.4 getpass

The `getpass` module reads input without displaying it normally on the terminal. The program uses `getpass.getpass()` in multiplayer mode, which helps prevent another player from immediately seeing the guess being typed.

6

Functions Created Before game()

The program follows a **modular programming** approach. Instead of one enormous block of code, individual responsibilities are separated into functions, and each performs a particular job.

6.1 generate_secret_number()

```
def generate_secret_number(low=1, high=100):  
    rand_num = random.randint(low, high)  
    return rand_num
```

Purpose: generates the secret number. **Parameters:** `low=1` and `high=100` are *default parameters*. Calling `generate_secret_number()` uses 1 and 100; calling `generate_secret_number(1, 500)` uses 1 and 500. **Return value:** `rand_num` is returned to the caller.

```
secret_number = generate_secret_number(1, 50)
```

6.2 compare_guess()

Compares `guess` with `secret_number`. It has three outcomes, which demonstrates **conditional branching**:

```
def compare_guess(guess, secret_number):  
    if guess > secret_number:  
        return "Too high! Try a lower number."  
    elif guess < secret_number:  
        return "Too low! Try a higher number."  
    else:  
        return "Correct"
```

The function *returns* a value instead of printing it. This separates the comparison logic from the presentation.

6.3 choose_difficulty()

Lets the player select Easy, Medium or Hard. It returns three values, for example `return 1, 50, 5`, meaning `low = 1, high = 50, max_attempt = 5`. These are returned as a **tuple** and can be **unpacked**:

```
low, high, max_attempt = choose_difficulty()
```

6.4 get_valid_guess()

One of the most important validation functions: `def get_valid_guess(low, high, mode):` It uses `while True:` because it must keep asking until the input is valid.

- Single-player: `guess = input(...)`
- Multiplayer: `guess = getpass.getpass(...)`
- Input arrives as a string such as "50"; `guess = int(guess)` turns it into the integer 50.
- The check `if low <= guess <= high:` is a **chained comparison**, equivalent to `low <= guess and guess <= high`.
- If valid: `return guess`. Otherwise the user is asked again.

6.5 Exception handling

The conversion `int(guess)` can fail: `abc` cannot become an integer, so Python would raise `ValueError`. The program handles it so invalid input cannot crash the game:

```
try:
    guess = int(guess)

except ValueError:
    print("Invalid input. Enter a whole number.")
```

6.6 calculate_score()

```
def calculate_score(attempts, max_attempt):
    score = ((max_attempt - attempts + 1) / max_attempt) * 100
    return round(score)
```

The fewer attempts the player uses, the higher the score. `round()` converts the decimal result into a rounded number.

6.7 get_rating()

Converts a numerical score into a star rating, which separates the scoring calculation from the presentation:

```
if score <= 20:
    return "★"
elif score <= 40:
    return "★★"
# ... and so on, up to five stars
```

6.8 choose_game_mode()

Asks the user to choose 1 (Single Player) or 2 (Multiplayer). The condition `if mode in ("1", "2"):` checks whether the entered value exists inside the tuple `("1", "2")`. Only valid choices are returned.

6.9 give_hint()

The hint system uses two strategies. When `wrong_guess == 3`, the program checks whether the secret number is even or odd using the **modulo operator** `%`, which returns the remainder of a division:

```
8 % 2 = 0      # even
9 % 2 = 1      # odd

if secret_number % 2 == 0:  # the number is even
```

Otherwise the function calculates a midpoint using **floor division** `//`:

```
midpoint = (low + high) // 2      # 10 // 3 = 3
```

The midpoint lets the program tell the player whether the secret number is in the lower or upper part of the range.

6.10 display_leaderboard()

Displays the leaderboard, which is a dictionary such as `{"John": 70, "Mary": 90}`. The steps are:

Step	Code	Effect
Convert to items	<code>leaderboard.items()</code>	Dictionary becomes sortable pairs.
Sort	<code>sorted(...)</code>	Orders the pairs.
Sort key	<code>key=lambda item: item[1]</code>	Sorts by the score (second element).
Reverse	<code>reverse=True</code>	Highest score first.

7

The game() Function

Purpose. `game()` is the main controller of the final application. It does not perform every individual operation itself; it coordinates the helper functions.

Overall structure

```
game()
├── Clear screen
├── Load/create leaderboard
├── Choose game mode
├── Determine number of players
├── Get player names
├── Repeat rounds
│   ├── Clear screen
│   ├── Choose difficulty
│   ├── Generate secret number
│   ├── Initialize game state
│   ├── Run guessing loop
│   ├── Determine winner
│   ├── Calculate score
│   ├── Update leaderboard
│   └── Show menu
```

Pseudocode

```

START game
Clear the terminal
Set leaderboard file
IF leaderboard file does not exist
    Create an empty leaderboard and save it to the file
ELSE
    Load leaderboard from file
Choose game mode
IF single player: number of players = 1 ELSE: set number of players
Get each player's name; validate; store

WHILE playing
    Clear screen; display new round
    Choose difficulty; display range and attempts
    Generate secret number
    Create attempt counter for each player
    Initialize hint counters
    Set winner to none; first player is current player

    WHILE nobody has won AND current player has attempts
        Display current player's turn
        Get a valid guess
        Increase current player's attempt count
        Compare guess with secret number; display result
        IF guess is correct
            Set current player as winner; display winner
            STOP guessing loop
        ELSE
            Record wrong guess
            IF all players have guessed incorrectly
                Increase full-round counter
                IF three full wrong rounds completed: display hint
            Move to next player

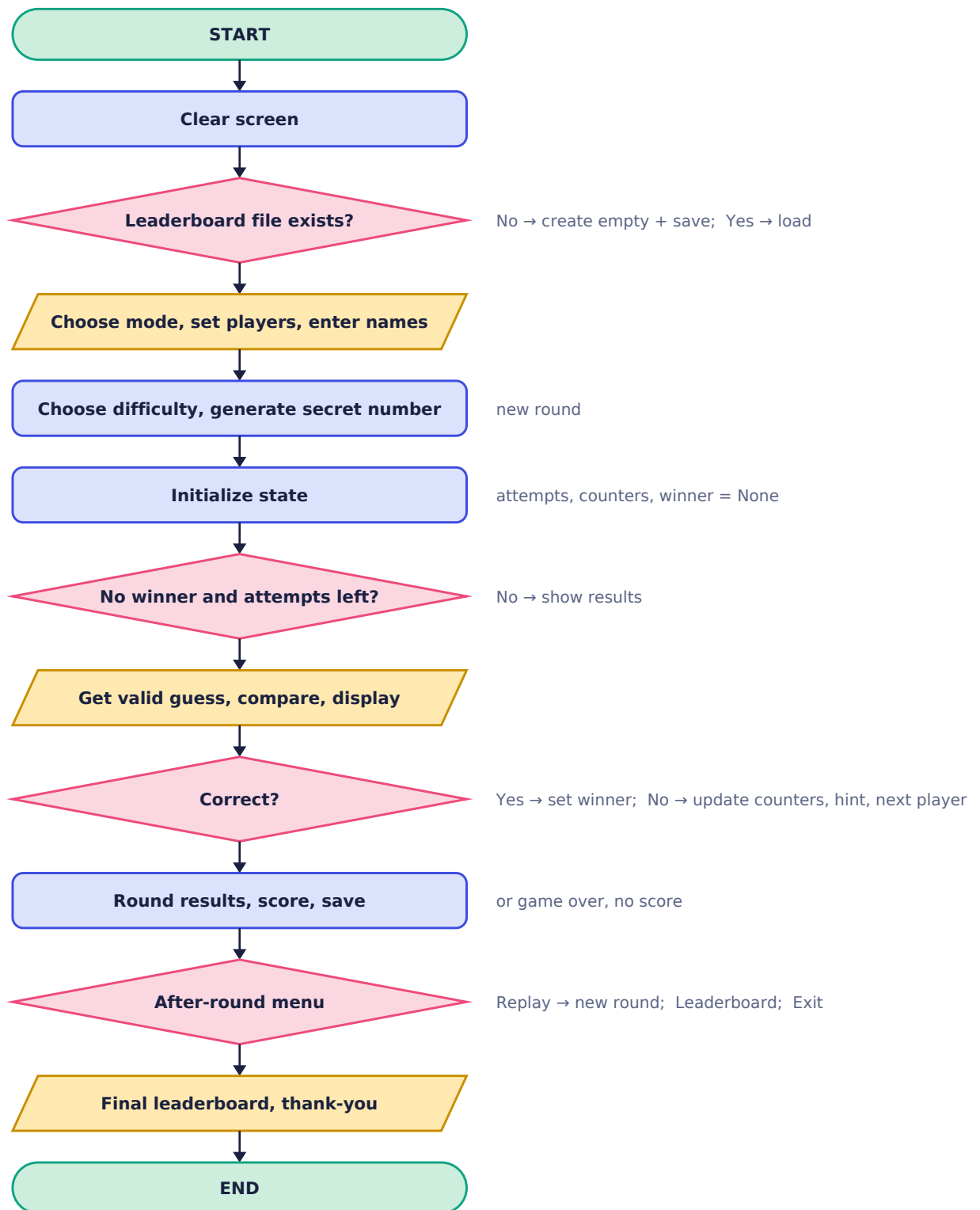
    Display round results
    IF there is a winner
        Calculate score and rating
        Update leaderboard; save leaderboard
    ELSE
        Display game-over message
    Display after-round menu
    IF play again: start another round
    ELSE IF leaderboard: display leaderboard
    ELSE IF exit: stop playing

Display final leaderboard
Display thank-you message
END game

```

Flowchart

Standard symbols: oval = start/end, rectangle = process, parallelogram = input/output, diamond = decision, arrows = flow.



8

Detailed Code Walkthrough

8.1 Clearing the screen

```
if os.name == "nt":
    os.system("cls")
else:
    os.system("clear")
```

`os.name` identifies the operating-system family: `"nt"` on Windows and commonly `posix` on Unix-like systems. The program picks the appropriate command so the command-line interface stays clean, clearing the terminal before each new section or round.

8.2 The leaderboard file

```
leaderboard_file = "leaderboard.json"

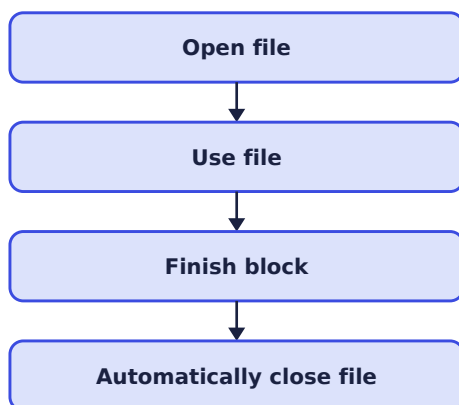
if not os.path.exists(leaderboard_file):
```

The first line stores the file name. `os.path.exists()` returns a **Boolean** (`True` or `False`) answering "does this file exist?". The `not` operator reverses the result, so the condition means: *if the file does NOT exist...*

8.3 with open() as file

```
with open(leaderboard_file, "w") as file:
```

This is a **context manager**. `open()` opens a file, and the mode `"w"` means *open for writing*. The file object is assigned to `file`. With `with`, Python automatically closes the file when the block finishes, which is safer and cleaner than manual closing.



8.4 JSON save and load

```
json.dump(
    leaderboard,
    file,
    indent=4
)

leaderboard = json.load(file)
```

`json.dump()` writes Python data into the file; `indent=4` keeps it readable. `json.load()` converts the JSON back into a Python object, which is why the leaderboard survives after the program closes.

8.5 Player management

```
player_names = []

for player_number in range(1, number_of_players + 1):
    ...
```

`player_names` is a **list**, an ordered collection. For one player `range(1, 2)` produces 1; for five players it produces 1, 2, 3, 4, 5. Each name is added with `player_names.append(name)`. The list preserves turn order, and an inner validation loop requires a non-empty, unique name:

```
while True:
    name = input(...).strip()
    if not name:
        print("Name cannot be empty.")
    elif name in player_names:
        print("That name is already taken.")
    else:
        player_names.append(name)
        break
```

8.6 .strip()

`input(...).strip()` removes whitespace from both ends: " Mathias " becomes "Mathias". This prevents accidental spaces from being stored as part of a name.

8.7 Round state

```
low, high, max_attempt = choose_difficulty()
secret_number = generate_secret_number(low, high)
attempts = [0] * number_of_players
wrong_guesses = 0
round_wrong_guesses = 0
winner = None
current_player = 0
```

List multiplication: with three players `[0] * 3` produces `[0, 0, 0]`, one counter per player by index.

Player	Index	Attempts so far
Player 1	0	2
Player 2	1	1
Player 3	2	3

winner = None. `None` represents the absence of a value, so nobody has won yet. When a player wins, `winner = current_player` holds that player's index.

current_player = 0. Python uses zero-based indexing: 0 is the first player, 1 the second, up to 4 for the fifth.

8.8 The main game loop

```
while (
    winner is None
    and attempts[current_player] < max_attempt
):
```

The loop continues only while two conditions are true: nobody has won, and the current player still has an attempt available.

8.9 Guess, compare, decide

```
guess = get_valid_guess(low, high, mode)
attempts[current_player] += 1
result = compare_guess(guess, secret_number)
print(result)

if result == "Correct":
    winner = current_player
    break
```

Validation, comparison and overall state control are deliberately separate. Invalid guesses never consume an attempt, because `get_valid_guess()` only returns once the input is a valid integer in range.

8.10 Wrong guesses and hints

```
round_wrong_guesses += 1
if round_wrong_guesses == number_of_players:
    wrong_guesses += 1
    round_wrong_guesses = 0
    if wrong_guesses % 3 == 0:
        print(give_hint(...))
```

Two counters separate an *individual* wrong guess from a *full cycle* in which every player guessed incorrectly. Hints are tied to full cycles (see Section 12).

8.11 Moving to the next player

```
current_player = (
    current_player + 1
) % number_of_players
```

Modulo wraps the index back to zero after the last player. With two players: $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$; with five: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$.

8.12 Results, scoring and persistence

After the turn loop the program checks whether a winner exists. If so, it calculates the winner's score and rating, updates the leaderboard **only when the new score is better**, and writes the dictionary back to JSON. If there is no winner, it displays the secret number and records no score.

8.13 After-round menu

Option	Action
1 — Play again	Breaks the menu loop and begins another round.
2 — View leaderboard	Displays the leaderboard, then returns to the menu.
3 — Exit	Sets <code>playing = False</code> , ending the outer round loop; the final leaderboard is shown.

9

Technical Concepts Used

Concept	Where and how it appears
Functions, parameters, arguments	Helpers receive values (low, high, mode, score); callers pass arguments such as <code>generate_secret_number(1, 500)</code> .
Return values	Helpers send results back, e.g. <code>compare_guess()</code> returns text instead of printing it.
Local variables / global constants	Variables like <code>secret_number</code> live inside functions; <code>MAX_PLAYERS</code> is defined once at the top.
Lists	<code>player_names</code> and <code>attempts</code> hold ordered collections.
Dictionaries	<code>leaderboard</code> maps player names to best scores.
Tuples and unpacking	<code>choose_difficulty()</code> returns three values assigned to three variables; <code>("1", "2")</code> is a tuple used for membership checks.
Indexing	<code>player_names[winner]</code> retrieves the winning name; indexes start at 0.
Loops, while True, break	Round loop, turn loop and validation loops; <code>break</code> exits a loop early.
Conditional branching	<code>if / elif / else</code> in comparison, menus, validation and scoring.
Exception handling	<code>try/except ValueError</code> around integer conversion.
File handling and JSON	<code>open()</code> , read/write modes, <code>with</code> , <code>json.dump()</code> and <code>json.load()</code> .
Booleans and None	Conditions evaluate to True/False; <code>winner = None</code> means no winner yet.
os features	<code>os.name</code> , <code>os.path.exists()</code> , <code>os.system()</code> .
Strings and f-strings	<code>f"Attempt: {attempt}"</code> embeds values in text; <code>.strip()</code> cleans input.
List multiplication	<code>[0] * number_of_players</code> builds one counter per player.
Modulo and floor division	<code>%</code> for even/odd, cyclic turns and every third wrong cycle; <code>//</code> for the midpoint.
Lambda, sorted(), enumerate()	Lambda is the sort key; <code>sorted()</code> orders entries; <code>enumerate()</code> adds ranking positions.
Functional-style elements	Small returning functions and lambda-based sorting, although the whole program is not purely functional.
State management	Attempts, turn, winner and playing variables describe the current state of the game.

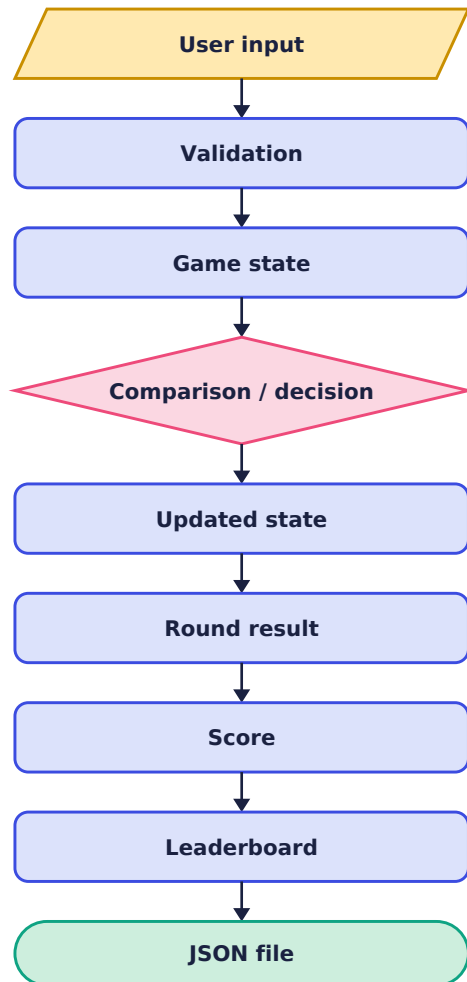
10

Game State and Data Flow

The Level 4 program is **state-driven**. State is information that changes as the game progresses.

Variable	Meaning	How it changes
<code>playing</code>	Whether another round should run	False when the user exits
<code>number_of_players</code>	Players in the current mode	Set before rounds
<code>player_names</code>	Names in turn order	Built before rounds
<code>low / high</code>	Current number range	Changed by difficulty
<code>max_attempt</code>	Current attempt limit	Changed by difficulty
<code>secret_number</code>	Hidden target	Generated each round
<code>attempts</code>	Attempts used by each player	Incremented after valid guesses
<code>wrong_guesses</code>	Completed full wrong cycles	Incremented after every player's wrong turn
<code>round_wrong_guesses</code>	Wrong guesses in the current cycle	Incremented, then reset
<code>winner</code>	Winner's index, or None	Set after a correct guess
<code>current_player</code>	Current turn index	Moves cyclically
<code>leaderboard</code>	Best saved score per player	Loaded, updated and saved

Data flow



11 Single-Player and Multiplayer: One Logic

The same turn loop handles both modes. Instead of writing separate code, the **number of players** controls how many turns happen per cycle.

Aspect	Single-player	Multiplayer
Players	<code>number_of_players = 1</code>	More than one (the current implementation sets 2)
Guess input	<code>input()</code>	<code>getpass.getpass()</code> so others cannot see the guess
Attempts	A list of one counter	One counter per player
Next player	$(0 + 1) \% 1 = 0$, always the same player	Rotates $0 \rightarrow 1 \rightarrow 0 \dots$
Wrong-cycle	Every wrong guess completes a cycle	Completed when all players have missed

Because turn rotation uses modulo, the same expression `(current_player + 1) % number_of_players` works for any number of players. This reduces duplication and makes the design scalable.

12 Hint System

Hints appear after every three *full* rounds of wrong guesses. A full round means every player has guessed incorrectly once.

Turn	Player	Result	round_wrong_guesses	wrong_guesses
1	Player 1	Wrong	1	0
2	Player 2	Wrong	2 = players → reset to 0	1
3-4	P1, P2	Wrong, Wrong	cycle complete	2
5-6	P1, P2	Wrong, Wrong	cycle complete	3 → hint

Which hint? `game()` requests a hint whenever `wrong_guesses % 3 == 0`. Inside `give_hint()`, the first hint (`wrong_guess == 3`) reports *even* or *odd*; later hints use the midpoint to narrow the range.

```
# Range 1-100, midpoint = (1 + 100) // 2 = 50
# secret_number <= 50 -> "between 1 and 50"
# secret_number > 50 -> "between 51 and 100"
```

13 Scoring and Rating System

The score rewards fewer attempts: `score = ((max_attempt - attempts + 1) / max_attempt) * 100`, rounded. A first-attempt win is always 100.

Attempts used	Easy (max 5)	Medium (max 7)
1	100	100
2	80	86
3	60	71
4	40	57
5	20	43
6	-	29
7	-	14

The rating then converts the score into stars: a score up to 20 gives one star, up to 40 gives two, and higher bands continue up to five stars. Example: a Medium win on attempt 6 scores 29, which falls in the two-star band. The game also prevents a player from exceeding the configured attempt limit.

14 Leaderboard System

- **Stored** in `leaderboard.json` as a dictionary mapping each name to their **best** score.
- **Created** as an empty dictionary the first time the program runs; **loaded** on later runs.
- **Updated** only when a winner's new score beats their saved score. A lower new score never replaces a higher one.
- **Displayed** sorted highest-first with ranking numbers from `enumerate()`.

```
{
    "Mary": 90,
    "John": 70
}
```

```
# Displayed as:
# 1. Mary - 90
# 2. John - 70
```

15

Level 1-3 Implementation Reference

These earlier versions are kept as reference. They show how each level adds one idea and reuses the helper functions.

Level 1

```
def level1():
    secret_number = generate_secret_number()
    guess = int(input("Guess a number between 1 and 100: "))
    result = compare_guess(guess, secret_number)
    print(result)
```

One complete cycle: generate → input → compare → display. No repeated guessing or attempt limit.

Level 2

```
def level2():
    secret_number = generate_secret_number()
    attempt = 0
    while True:
        guess = int(input("Guess a number between 1 and 100: "))
        attempt += 1
        result = compare_guess(guess, secret_number)
        print(f"{result}. Attempt: {attempt}")
        if guess == secret_number:
            print(f"You win. Total attempts: {attempt}")
            break
```

Introduces repetition with `while True`, an attempt counter, and `break` to stop on the correct guess.

Level 3

```
def level3():
    low, high, max_attempt = choose_difficulty()
    print(f"You have {max_attempt} attempts.")
    secret_number = generate_secret_number(low, high)
    attempt = 0
    while attempt < max_attempt:
        guess = get_valid_guess(low, high)
        attempt += 1
        result = compare_guess(guess, secret_number)
        print(result)
        if result == "Correct":
            print(f"Congratulations! You got it in {attempt} attempts.")
            break
        if attempt == max_attempt:
            print(f"Game Over! Secret number was: {secret_number}")
        else:
            print(f"You have {max_attempt - attempt} attempts left.")
```

Adds difficulty selection, strict validation, a maximum attempt count and game-over handling.

Level	Main change
1	One guess + comparison.
2	Repeated guesses + attempt counter.
3	Difficulty + validation + attempt limit.
4	Hints + scoring + rating + multiplayer + rounds + leaderboard + persistence.

16

Important Questions and Answers

Q1. Why divide the program into functions?

To make it modular, reusable and easier to understand, test and debug. Each function has one responsibility.

Q2. Why does `compare_guess()` return text instead of printing?

Returning a value separates comparison logic from presentation, so the caller decides what to do with the result.

Q3. Why use `while True` in `get_valid_guess()`?

We don't know how many attempts the user needs to enter valid input, so the loop continues until a valid guess is returned.

Q4. Why use `try/except`?

Converting text such as 'abc' with `int()` raises `ValueError`. Catching it prevents a crash and lets the program re-ask.

Q5. Why are modes compared with "1" and "2" but guesses converted with `int()`?

`input()` always returns a string. Menu choices can be compared as strings; guesses need to be integers for numeric comparison.

Q6. Do invalid guesses cost an attempt?

No. `get_valid_guess()` only returns after the input is a valid integer inside the selected range.

Q7. Why use `with open()`?

It closes the file automatically when the block ends, even if something goes wrong, which is safer than manual closing.

Q8. Why is the leaderboard stored in JSON?

JSON is a simple text format that maps naturally to Python dictionaries and persists data between runs.

Q9. How does turn rotation work for any number of players?

`current_player = (current_player + 1) % number_of_players` wraps the index back to zero after the last player.

Q10. Does the game support 1-5 players?

The architecture can, but the current code sets `multiplayer` to exactly two players. `MAX_PLAYERS` (5) is defined but unused; a player-count function would extend it.

Q11. When does a hint appear and which type?

After every third full wrong cycle. The first hint is even/odd; later ones narrow the range with the midpoint.

Q12. Does a lower score replace the saved best score?

No. The leaderboard stores each player's best score only.

Q13. Does defining `game()` run it?

No. Defining a function only creates it. The final call to `game()` starts the application.

17

Conclusion

The Number Guessing Game shows how a simple Python exercise becomes a structured command-line application. It progresses from random number generation and comparison to loops, validation, difficulty levels, hints, scoring, multiplayer turns, multiple rounds and persistent storage.

The central `game()` function coordinates the application while helper functions handle focused tasks. This separation makes the code easier to explain, test, debug and extend.

Key lesson

Complex behaviour can be built from smaller, understandable components. The main data flow is: user input → validation → game state → comparison → decision → score → leaderboard → persistent storage.